CSC291 – Software Engineering Concepts
Lecture 11 & 12: Requirements Modelling

1. Requirements Analysis
Requirements analysis specifies a software's operational characteristics, indicates its interfaces with other system elements, and establishes constraints the software must meet. It allows the analyst/modeler to:

- Elaborate on basic requirements established during earlier RE tasks.
- Build models depicting user scenarios, functional activities, problem classes, system behavior, and data flow.

2. Rules of Thumb for Analysis Models
- Focus on requirements visible within the problem/business domain — keep abstraction high.
- Each model element should add to understanding of requirements (information, function, behavior).
- Delay infrastructure and non-functional concerns until design.
- Minimize coupling throughout the system.
- Ensure the model provides value to all stakeholders.
- Keep the model as simple as possible.

3. Domain Analysis
Identification, analysis, and specification of common requirements from a specific application domain, typically for reuse across multiple projects.

Steps:

- Define the domain to be investigated.
- Collect a representative sample of applications in the domain.
- Analyze each application in the sample.
- Develop an analysis model for the objects.

Sources of domain knowledge: Technical literature, existing applications, customer surveys, expert advice, current/future requirements.

Outputs: Class taxonomies, reuse standards, functional models, domain languages.

4. Requirement Modelling — Overview
Provides a description of the required **informational**, **functional**, and **behavioral** domains for a computer-based system. The model changes dynamically as understanding grows.

Four types of models are used together to view requirements from different perspectives:

- **Scenario-based models** — e.g., use cases, user stories
- **Class-based models** — e.g., class diagrams, collaboration diagrams
- **Flow models** — e.g., DFDs, data models
- **Behavioral models** — e.g., state diagrams, sequence diagrams

5. Scenario-based Modelling (Use Cases)

Use cases describe how the system will be used. Each tells a stylized story of how an **actor** (end user or external entity) interacts with the system under specific circumstances.

The story can be narrative text, a task outline, a template-based description, or a diagram.

Defining Actors

Actors are people or devices that interact with the software. A single person may play multiple actor roles, or multiple people may share one role.

Example (ATM System): Bank Customer (primary actor) interacts with Withdraw Money, Deposit Money, Transfer Money, and Check Balance. Customer Accounts Database is a secondary actor.

Use Case Questions

Each use case scenario should answer:

- Who is the primary actor? Who are the secondary actors?
- What are the actor's goals?
- What preconditions must exist before the story begins?
- What main tasks or functions does the actor perform?
- What system information will the actor acquire, produce, or change?
- What variations in the actor's interaction are possible?
- Does the actor need to be informed about unexpected changes?

6. Class-based Modelling

Shows the **classes** of a system, their inter-relationships, and the operations and attributes of each class. Used to explore domain concepts and analyze requirements.

A class-based model represents:

- Objects the system will manipulate.
- Operations (methods/services) applied to those objects.
- Relationships (including hierarchical ones) between objects.
- Collaborations between defined classes.

Identifying Classes

Perform a **grammatical parse** of usage scenarios: underline each noun or noun phrase. Synonyms are noted. Determine if the class belongs to the **solution space** (needed to implement) or **problem space** (needed to describe).

Class Structure (UML)

A class is rendered as a rectangle with three compartments:

- **Top:** Class name (required)
- **Middle:** Attributes — named properties describing the object (e.g., name: String, birthdate: Date)
- **Bottom:** Operations — behaviors of the class (e.g., eat, sleep, work)

Example: Class **Person** inherits into **Student** and **Professor**. Student has attributes studentNumber and averageMark; Professor has salary and yearsOfService. A Professor supervises 0..* Students.

7. Behavioural Modelling (State Diagrams)

A **state diagram** depicts the data and behavior of a single object throughout its lifetime. The entire diagram is drawn from that object's perspective.

Elements of a State Diagram

- **States:** Conceptual descriptions of the object's data at a point in time, represented by field values. Only include states that are conceptually different.
- **Transitions:** Arrows showing movement between states, triggered by events/conditions.
- **Initial pseudostate:** The starting point (filled circle).
- **Final state:** The end point (filled circle with border).

Example — Safe Scenario

States: **Wait** → **Lock** → **Open**

- Wait → Lock: candle removed [door closed] / reveal lock
- Lock → Open: key turned [candle in] / open safe
- Open → Wait: safe closed
- Lock → Final: key turned [candle out] / release killer rabbit

Key Points

- Requirements modelling uses four views: scenario-based, class-based, flow, and behavioral.
- Use cases identify actors and their goals; a complete set covers all system interactions.
- Class diagrams are identified via grammatical parsing of scenarios (nouns = classes).
- State diagrams model a single object's lifetime from its own perspective.
- Models are dynamic — they evolve as understanding of requirements improves.

Reading: Chapter 6: Requirement Modelling — Software Engineering: A Practitioner's Approach | Resource: https://sites.google.com/cuilahore.edu.pk/sec